Signals and Systems Project

Phase 1

Sharif University of Technology

An Audio-Based Emotion Classification System Using Machine Learning Algorithms and Feature Engineering

---

Note that this report has been done using LaTeX.

Instructor: Prof. Behrouzi

Sharif University of Technology,
Electrical Engineering Faculty

Winter 2026

Table of Contents

1 Preface . . . 3
2 Exploratory Data Analysis . . . 4
2.1 Importing the Necessary Libraries . . . 4
2.2 Loading an Audio Sample . . . 4
3 Feature Engineering . . . 6
3.1 Framing the Audio Signals . . . 6
3.2 Amplitude Normalization . . . 6
3.3 Short-Time Energy Calculation . . . 7
3.4 Removing the Low-Energy Frames . . . 7
3.5 Calculating the Zero Crossing Rate (ZCR) . . . 7
3.5.1 What is ZCR? . . . 7
3.6 Applying the Hamming Window to our Frames . . . 8
3.6.1 What is Hamming Window? . . . 8
3.7 Fundamental Frequency Estimation . . . 9
3.8 Fourier Transform Computation . . . 10
3.9 Computing the Spectral Centroid . . . 11
3.10 Computing the Spectral Bandwidth . . . 11
3.11 Computing the Spectral Flux . . . 12
3.12 Computing the MFCCs . . . 12
3.12.1 What are MFCCs? . . . 12
4 Data Preparation for Model Training . . . 15
4.1 Extracting the Classification Labels . . . 16
5 Reading the Dataset . . . 17
5.1 Splitting the Dataset . . . 17
6 Training the Machine Learning Models . . . 18
6.1 LDA (Linear Discriminant Analysis) . . . 18
6.1.1 Key Insights . . . 20
6.2 SVM (No Kernel) . . . 20
6.2.1 Key Insights . . . 22
6.3 SVM (RBF Kernel) . . . 22
6.3.1 Key Insights . . . 24
7 The Problem . . . 25
7.1 What is Data Augmentation? . . . 25
8 Class Reduction . . . 27
8.1 Key Insights . . . 28
9 Dimension Reduction . . . 29
9.1 Key Insights . . . 30

10 Data Augmentation . . . 31
10.1 Stretch in the Time Domain . . . 31
10.2 Adding AWGN Noise . . . 31
10.2.1 What is AWGN Noise? . . . 31
10.3 Adjusting the Audio Signal Volume . . . 32
11 Re-training the Machine Learning Models . . . 35
11.1 Key Insights . . . 35
12 Tuning Model Parameters . . . 37
12.1 What is Grid Search? . . . 37
12.2 What is Cross Validation (CV)? . . . 37
12.3 Key Insights . . . 38
13 Generalization . . . 39
13.1 Key Insights . . . 40
13.1.1 What is Feature Distribution Mismatch? . . . 40
13.2 Key Insights . . . 40
13.2.1 Key Insights . . . 41
14 Conclusion . . . 43

1 Preface

This project was done using **Python**, utilizing machine learning and signal processing libraries such as **NumPy** and **scikit-learn**.

Note that no advanced audio signal processing libraries have been used in this project, and all of the methods including feature engineering and data augmentation have been implemented using low-level signal processing libraries, in order to provide a deeper understanding of the underlying signal processing principles and machine learning pipeline.

There will be a **Jupyter Notebook** attached to this document containing all of the Python code and analysis results of this project.

2 Exploratory Data Analysis

To train our machine learning models, we are using the **RAVDESS** dataset, which consists of 1440 speech audio files performed by 24 actors.

The dataset consists of 8 different emotion classes.

One of the minor challenges that we faced in this project was that some of the audio files were mono-channel while others were stereo.

In order to have a homogeneous set of data points, we averaged the 2 channels in stereo audio files and converted them into mono-channel audio signals.

2.1 Importing the Necessary Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.io import wavfile
import os
from scipy.fftpack import dct

from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA

from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import accuracy_score, confusion_matrix

from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.model_selection import GridSearchCV

warnings.filterwarnings("ignore", category=WavFileWarning)
```

2.2 Loading an Audio Sample

```
Fs, original_audio = wavfile.read("sample.wav")

plt.plot(np.linspace(0, original_audio.shape[0], original_audio.shape[0]), original_audi
plt.title('Sample Audio File Waveform')
plt.xlabel('Sample')
plt.ylabel('Magnitude')
```

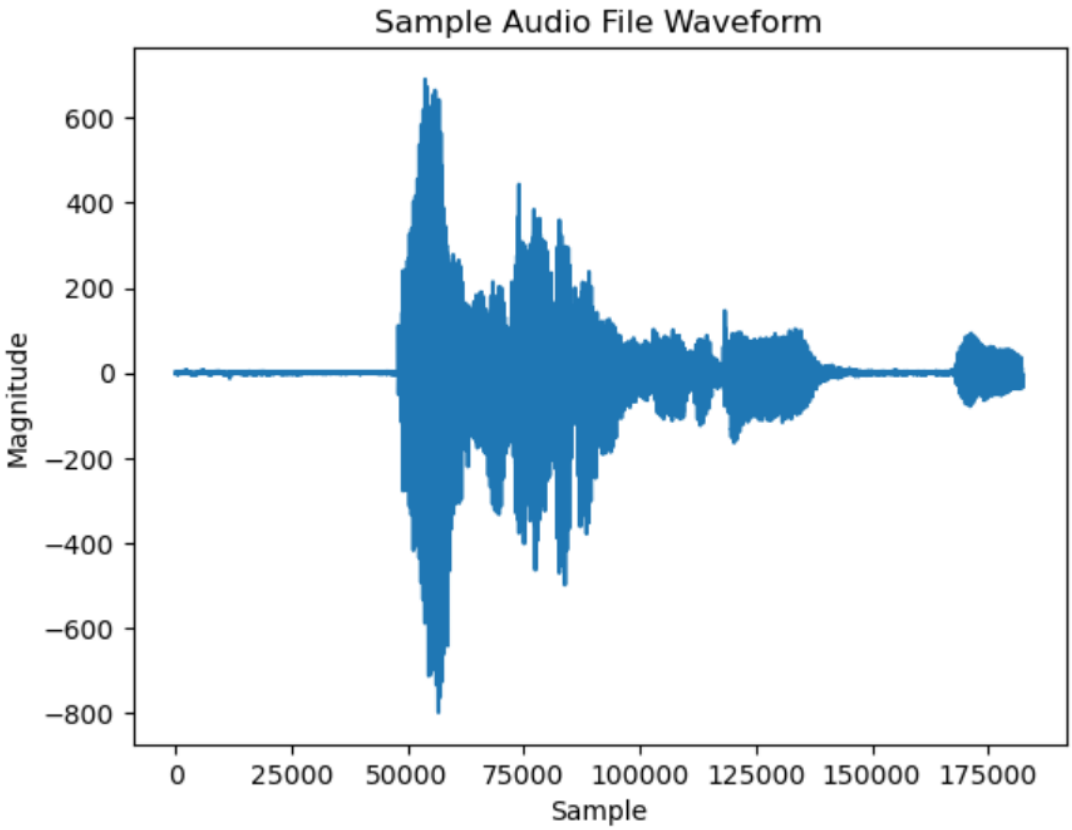

Figure 1: A Sample Audio File Waveform from the Dataset

There are a few key points to consider about this sample data from the dataset.

- It seems that the audio files are not trimmed completely to include just the speech part. There are silent and low energy periods present. This will definitely be a problem when extracting the desired features to train our machine learning model.
- Even though all of the audio files have been recorded in a controlled studio environment by professional voice actors, there are some inconsistencies in the data points, such as audio channel configuration, volume range, etc. Such inconsistencies will definitely need proper normalization in the feature extraction process.

3 Feature Engineering

3.1 Framing the Audio Signals

To be able to accurately extract the desired features to train our machine learning models, we will have to split our audio signals into overlapping frames (epochs).

The problem with extracting all of the features from the entire signal is that the silent and low energy parts will cause problems in accurately extracting the features.

This will assist us in achieving a better insight into what the actual features are.

I have chosen the frame size to be 25 milliseconds with an overlap value of 15 milliseconds.

```
frame_size_milliseconds = 25
hop_size_milliseconds = 10
frame_size = int(frame_size_milliseconds * 0.001 * Fs)
hop_size = int(hop_size_milliseconds * 0.001 * Fs)

def get_frames(audio_file):
    frames = []
    n_samples = audio_file.shape[0]

    start = 0

    while start < n_samples:
        end = start + frame_size

        frame = audio_file[start:end]
        if len(frame) < frame_size:
            frame = np.pad(frame, (0, frame_size - len(frame)))

        frames.append(frame)
        start += hop_size

    return frames
```

3.2 Amplitude Normalization

As stated before, we need our data to be homogeneous in order to have an unbiased machine learning model.

One of the most important things to consider is to normalize o audio signals amplitude by dividing the amplitudes all by the maximum amplitude available in the audio signal.

Not doing such will definitely affect the feature extraction process and bias our machine learning model.

```
def normalize_frames(frames, maximum_amplitude):
    normalized_frames = [frame.astype(np.float32) / maximum_amplitude for frame in frame
    return normalized_frames
```

3.3 Short-Time Energy Calculation

One of the important features to be considered, is the energy of the signal.

Some emotions like *Surprise* or *Happy* definitely have more high energy frames than emotions like *Calm* or *Sad*.

I calculate the energy frame for each of our overlapping frames one by one using the following formula:

$$\epsilon\{x\} = \Sigma_{k=0}^{N-1} x^2[k]$$

```
def short_time_energy(frames):
    energies = []

    for frame in frames:
        energies.append((frame ** 2).sum() / frame.shape[0])

    return np.array(energies)
```

3.4 Removing the Low-Energy Frames

Since there are many silent and low-energy periods and frames present in the audio signal, I will remove the low-energy frames to get an unbiased set of features to train out machine learning model.

Frames with less than 10% of the maximum energy among all of the frames will be removed.

```
def remove_low_energy_frames(frames):
    energies = short_time_energy(frames)
    maximum_energy = energies.max()
    threshold = 0.1 * maximum_energy

    final_frames = [frame for frame, energy in zip(frames, energies) if energy >= thresh
    final_energies = [energy for frame, energy in zip(frames, energies) if energy >= thr

    return final_frames, np.array(final_energies)
```

3.5 Calculating the Zero Crossing Rate (ZCR)

3.5.1 What is ZCR?

Zero Crossing Rate or **ZCR** determines how many times the amplitude of the audio signal has gone from a positive value to a negative value.

This is an important feature to be considered, because higher ZCR indicates more variation in the amplitude of the audio signal which might be an indicator of a more energetic emotion.

```
def zcr(final_frames):
    zcrs = []
```

```
    for frame in final_frames:
        zc = np.sum(np.abs(np.diff(np.sign(frame)))) / (2 * len(frame))
        zcrs.append(zc)

    return np.array(zcrs)
```

3.6 Applying the Hamming Window to our Frames

From now on, we will be focusing on the frequency domain features of these audio signals, which will require computing the FFT (Fast Fourier Transform) of these signals.

3.6.1 What is Hamming Window?

Since we will be computing the FFT for each audio signal frame separately, each frame will be dealt with like an individual audio signal.

When splitting the original audio signal into frames, we introduce discontinuities at the frame boundaries, which is not one of the characteristic of a normal audio signal.

Audio signals naturally start and end gradually, not suddenly.

To avoid inconsistent computation of FFT and amplifying frequencies that appear because of the discontinuities present at the frame boundaries, we use the **Hamming Window**.

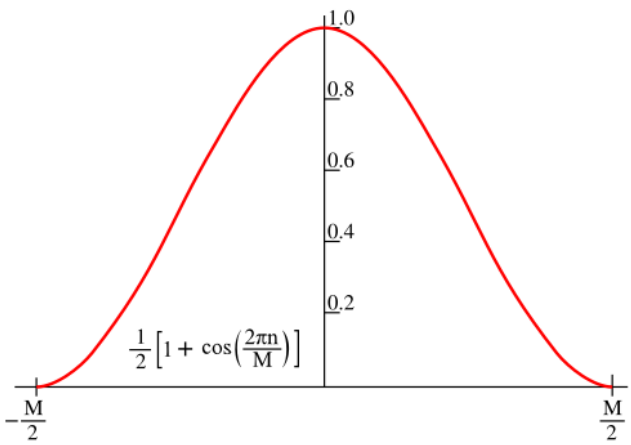

Figure 2: Hamming Window

$$H(n) = \frac{1}{2}\left[1 + \cos(\frac{2\pi n}{M})\right]$$

Hamming Window smooths out the boundaries of the audio signal frames which assists us in achieving a better frequency spectrum analysis of each frame.

In fact, since a normal frame without applying any window will have discontinuities at the boundaries of the frame, this will introduce frequencies in the entire spectrum with high amplitudes.

Using a hamming window, these high frequencies that are introduced because of our rectangular framing will vanish.

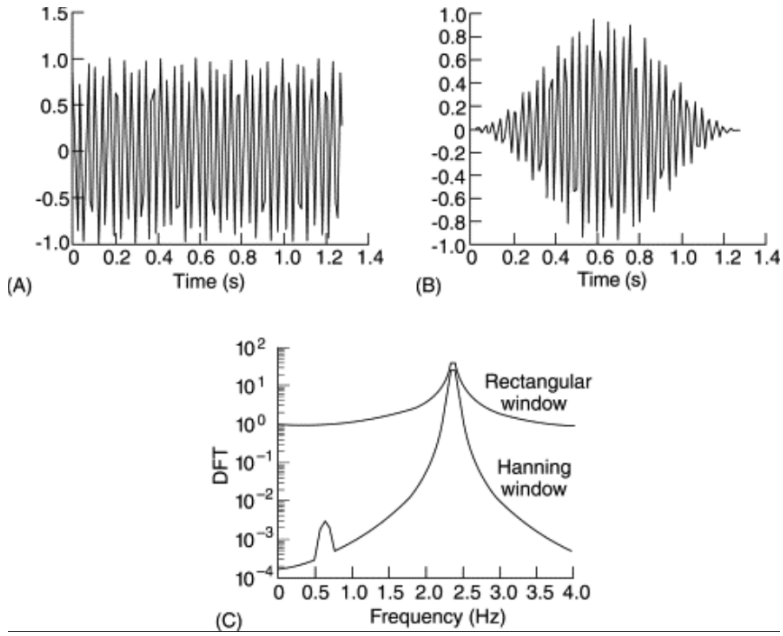

Figure 3: Hamming Window applied to an Audio Signal

```
def hamming_windowed_frames(final_frames):
    frame_size = final_frames[0].shape[0]
    n = np.arange(frame_size)
    hamming_window = 0.54 - 0.46 * np.cos((2 * np.pi * n) / (frame_size - 1))

    final_frames_windowed = [hamming_window * frame for frame in final_frames]

    return final_frames_windowed
```

3.7 Fundamental Frequency Estimation

In this part, we are going to be using mathematical concepts to estimate the fundamental frequency (pitch) for each of our audio signal frames.

Since audio signals are somehow periodic (Natural Audio Signals), we can say:

$$x[n] \approx x[n - T_0]$$

Since we are dealing with DT (Discrete Time) audio signals, we have:

$$f_0 = \frac{F_s}{lag}$$

To be able to estimate the value of T_0 and f_0, we use the concept of autocorrelation.

Autocorrelation moves the digital signal in the time axis one sample by one sample and multiplies it by the original signal.

If the shifting value is equal to T_0, the multiplication of the original signal and it's shifted value will be maximum.

$$R[k] = \Sigma_{n=0}^{N-1-k} x[n]x[n+k]$$

So the function $R[k]$ should peak at $k \in \{0, T_0, 2T_0, ...\}$.

So after removing the DC offset from the signal (converting the signal into a zero-DC signal), we compute the autocorrelation function for each of the audio signal frames.

Since we are only interested in frequencies produced by humans vocal cords, we limit ourselves to check only the lagging values corresponding to the frequency range of $f \in [80Hz, 300Hz]$.

We also remove the correlation values lower than 30 percent of the maximum correlation value ($k = 0$) to get a more unbiased result.

After that, we find the lag value corresponding to the highest correlation value and compute the frequency using the formula stated before.

```
def fundamental_frequency(final_frames_windowed, Fs):
    fundamental_frequencies = []

    fmin = 80
    fmax = 300

    for frame in final_frames_windowed:
        frame = frame - np.mean(frame)

        corr = np.correlate(frame, frame, mode='full')
        corr = corr[len(corr)//2:]

        lag_min = int(Fs / fmax)
        lag_max = int(Fs / fmin)

        corr_range = corr[lag_min:lag_max]

        peak_index = np.argmax(corr_range)
        true_lag = peak_index + lag_min

        if corr[true_lag] < 0.3 * corr[0]:
            f0 = 0
        else:
            f0 = Fs / true_lag

        fundamental_frequencies.append(f0)

    return fundamental_frequencies
```

3.8 Fourier Transform Computation

From now on, we will be directly extracting audio signal features from the frequency spectrum, so we need to calculate the Fourier Transform for each of our frames.

There are several methods to calculate the frequency spectrum of a given signal. We could have implemented the **Discrete-time Fourier Transform** using pure Python, but FFT (Fast

Fourier Transform) is a much more efficient algorithm for calculating the fourier transform of discrete signals.

Since we have a large dataset of audio signals, we decided to use the more efficient FFT algorithm.

```
def fft(final_frames_windowed):
    fft_frames = []

    for frame in final_frames_windowed:
        spectrum = np.fft.rfft(frame)
        magnitude = np.abs(spectrum)
        fft_frames.append(magnitude)

    return fft_frames
```

3.9 Computing the Spectral Centroid

Spectral Centroid can be interpreted as the weighted sum of all the frequencies available in a signal, or the center of mass in the frequency spectrum.

$$C = \frac{\Sigma_{k=0}^{N-1} f_k |X(k)|}{\Sigma_{k=0}^{N-1} |X(k)|}$$

The denominator normalizes the calculated centroid.

```
def spectral_centroid(fft_frames, Fs):
    centroids = []
    N = (len(fft_frames[0]) - 1) * 2

    freqs = np.fft.rfftfreq(N, 1/Fs)

    for magnitude in fft_frames:
        centroid = np.sum(freqs * magnitude) / np.sum(magnitude)
        centroids.append(centroid)

    return centroids
```

3.10 Computing the Spectral Bandwidth

The Spectral Bandwidth tells us how separated the frequencies are around the centroid for each frame.

It's like calculating the variance for a set of data points, and the formula for it's computation verifies that (it's so similar to the formula for variance).

$$B = \sqrt{\frac{\sum_{k=0}^{N-1} (f_k - C)|X_k|}{\sum_{k=0}^{N-1} |X_k|}}$$

```python
def spectral_bandwidth(fft_frames, centroids, Fs):
    bandwidths = []
    N = (len(fft_frames[0]) - 1) * 2

    freqs = np.fft.rfftfreq(N, 1/Fs)

    for mag, centroid in zip(fft_frames, centroids):
        bw = np.sqrt(np.sum(mag * (freqs - centroid)**2) / np.sum(mag))
        bandwidths.append(bw)

    return bandwidths
```

3.11 Computing the Spectral Flux

The Spectral Flux tells us how the frequency distribution changes between 2 consecutive frames.

Since the frames are highly overlapping, this is a pretty useful feature to consider.

```python
def spectral_flux(fft_frames):
    fluxes = []

    for i in range(1, len(fft_frames)):
        diff = fft_frames[i] - fft_frames[i-1]
        fluxes.append(np.sum(diff**2))

    return fluxes
```

3.12 Computing the MFCCs

3.12.1 What are MFCCs?

Mel-Frequency Cepstral Coefficients or **MFCCs** are one of the most widely used features in speech and audio processing.

They provide a compact representation of the spectral envelope of an audio signal, reflecting how humans perceive sound.

The computation of MFCCs consists of the following steps:

1. **Fourier Transform:** Compute the magnitude spectrum of each frame using the **Discrete Fourier Transform (DFT)**:

$$X_m[k] = \sum_{n=0}^{N-1} x_m[n] e^{-j2\pi kn/N}, \quad 0 \leq k < N$$

2. **Mel Filterbank:** Map the linear frequency spectrum to the *Mel scale* using triangular filters:

$$m = 2595 \cdot \log_{10}\left(1 + \frac{f}{700}\right)$$

Then compute the log-energy in each Mel filter:

$$S_m[i] = \log\left(\sum_k |X_m[k]|^2 \cdot H_i[k]\right)$$

where $H_i[k]$ is the i-th Mel filter.

3. **Discrete Cosine Transform (DCT):** Apply the DCT to decorrelate the log-Mel energies and obtain the MFCC coefficients:

$$c_m[n] = \sum_{i=0}^{K-1} S_m[i] \cos\left[\frac{\pi n}{K}\left(i + \frac{1}{2}\right)\right], \quad 0 \le n < L$$

where L is the number of retained MFCCs.

Key Points:

- MFCCs capture the perceptually important aspects of the audio spectrum while reducing dimensionality.
- The first coefficient (c_0) corresponds to the overall signal energy, while higher coefficients encode spectral shape.
- MFCCs are often used with their first- and second-order derivatives (delta and delta-delta) to capture temporal dynamics.

```
def hz_to_mel(hz):
        return 2595 * np.log10(1 + hz / 700)

def mel_to_hz(mel):
        return 700 * (10**(mel/2595) - 1)

def mfcc(frames, Fs):
    n_fft=512
    num_ceps=13
    n_filters=26
    mfccs = []

    for frame in frames:
        spectrum = np.abs(np.fft.rfft(frame, n=n_fft))**2

        mel_points = np.linspace(hz_to_mel(0), hz_to_mel(Fs/2), n_filters+2)
        hz_points = mel_to_hz(mel_points)
        bin_points = np.floor((n_fft+1) * hz_points / Fs).astype(int)

        fbank = np.zeros((n_filters, n_fft//2+1))

        for m in range(1, n_filters+1):
```

```
            f_m_minus = bin_points[m-1]
            f_m = bin_points[m]
            f_m_plus = bin_points[m+1]
            for k in range(f_m_minus, f_m):
                fbank[m-1, k] = (k - f_m_minus) / (f_m - f_m_minus)
            for k in range(f_m, f_m_plus):
                fbank[m-1, k] = (f_m_plus - k) / (f_m_plus - f_m)

        filter_banks = np.dot(fbank, spectrum[:n_fft//2+1])
        filter_banks = np.where(filter_banks == 0, np.finfo(float).eps, filter_banks)  #

        log_fbanks = np.log(filter_banks)

        mfcc = dct(log_fbanks, type=2, norm='ortho')[:num_ceps]
        mfccs.append(mfcc)

    return np.array(mfccs)
```

4 Data Preparation for Model Training

Now that we have all of the functionalities necessary for extracting the features for each audio file in our dataset, we have to aggregate our features into a Python Dictionary to be able to feed it into our model.

The thing is that we have calculated these features for each frame separately. We obviously cannot feed all of the features for each frame into the model, because different audio files have different number of frames, and by doing this, we are not getting any statistical insight from the features we calculated, and the number of features will be so large that can cause the model training to be so slow and cause the model to overfit.c

So we will compute the statistical components for each of the features calculated for each frame.

```
def get_aggregated_features(audio_file, Fs):
    features = {}

    frames = get_frames(audio_file)
    maximum_amplitude = np.max(np.abs(audio_file))
    frames = normalize_frames(frames, maximum_amplitude)

    final_frames, final_energies = remove_low_energy_frames(frames)
    zcrs = zcr(final_frames)
    final_frames_windowed = hamming_windowed_frames(final_frames)
    fundamental_frequencies = fundamental_frequency(final_frames_windowed, Fs)
    ffts = fft(final_frames_windowed)
    centroids = spectral_centroid(ffts, Fs)
    bandwidths = spectral_bandwidth(ffts, centroids, Fs)
    fluxes = spectral_flux(ffts)
    mfccs = mfcc(final_frames_windowed, Fs)
    delta_1s = mfcc_delta(mfccs)
    delta_2s = mfcc_delta(delta_1s)

    for name, values in zip(
        ['energy', 'zcr', 'f0', 'centroid', 'bandwidth', 'flux'],
        [final_energies, zcrs, fundamental_frequencies, centroids, bandwidths, fluxes]
    ):
        features[name + '_mean'] = np.mean(values)
        features[name + '_std']  = np.std(values)
        features[name + '_min']  = np.min(values)
        features[name + '_max']  = np.max(values)

    for i in range(mfccs.shape[1]):
        features[f'mfcc_{i+1}_mean'] = np.mean(mfccs[:, i])
        features[f'mfcc_{i+1}_std'] = np.std(mfccs[:, i])
        features[f'mfcc_{i+1}_min'] = np.min(mfccs[:, i])
        features[f'mfcc_{i+1}_max'] = np.max(mfccs[:, i])
```

```
    for i in range(delta_1s.shape[1]):
        features[f'delta1_{i+1}_mean'] = np.mean(delta_1s[:, i])
        features[f'delta1_{i+1}_std'] = np.std(delta_1s[:, i])
        features[f'delta1_{i+1}_min'] = np.min(delta_1s[:, i])
        features[f'delta1_{i+1}_max'] = np.max(delta_1s[:, i])

    for i in range(delta_2s.shape[1]):
        features[f'delta2_{i+1}_mean'] = np.mean(delta_2s[:, i])
        features[f'delta2_{i+1}_std'] = np.std(delta_2s[:, i])
        features[f'delta2_{i+1}_min'] = np.min(delta_2s[:, i])
        features[f'delta2_{i+1}_max'] = np.max(delta_2s[:, i])

    return features
```

4.1 Extracting the Classification Labels

To be able to train and evaluate our machine learning model, we will have to extract the classification labels from the audio files in the dataset, which will indicate the emotion they represent.

To do such, we will have to read the file names and extract the emotion (classification label) using the codes in the file name.

```
def extract_emotion_label(file_name):
    parts = os.path.basename(file_name).split('-')
    emotion_code = int(parts[2])

    emotion_map = {
        1: 'neutral',
        2: 'calm',
        3: 'happy',
        4: 'sad',
        5: 'angry',
        6: 'fear',
        7: 'disgust',
        8: 'surprise'
    }

    return emotion_map[emotion_code]
```

5 Reading the Dataset

In this part, we read the audio files in the dataset one by one, extract the desired features using the already implemented functions, aggregate all of the features into a **Python Dictionary**, and save them in a Python list.

```
files = []

folder_path = './dataset'
all_file_names = [os.path.join(folder_path, f) for f in os.listdir(folder_path)]
count = 0
all_count = len(all_file_names)
progress_name = 'Reading Data'

for file_name in all_file_names:
    Fs, original_audio = wavfile.read(file_name)

    if original_audio.ndim == 2:
        original_audio = original_audio.mean(axis=1)

    file = get_aggregated_features(original_audio, Fs)
    file['file_name'] = file_name
    file['emotion'] = extract_emotion_label(file_name)

    files.append(file)

    count += 1
```

5.1 Splitting the Dataset

The standard method for training and evaluating a machine learning model is to split our dataset into training and testing sets, and only use the training set to train our machine learning model.

Using this method, the testing data will not be exposed to the model at all and we will have a much more reliable evaluation of the accuracy of the model.

We decided to use 20 percent of our entire dataset just for model evaluation.

```
features = [feature for feature in files[0].keys() if feature != 'file_name' and feature

X = [[file[k] for k in features] for file in files]
y = [file['emotion'] for file in files]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

6 Training the Machine Learning Models

6.1 LDA (Linear Discriminant Analysis)

This machine learning algorithm, tries to find the linear combination of features that best separates the classes.

LDA also performs dimension reduction, but it's not as effective as algorithms like **PCA**.

Unlike the PCA algorithm which tries to maximize the variance, the LDA algorithm is optimized to maximize the class separability.

After projecting the features into a lower dimension, this algorithm can be used to classify data.

```
lda = LinearDiscriminantAnalysis()
lda.fit(X_train, y_train)
y_pred_test = lda.predict(X_test)
y_pred_train = lda.predict(X_train)
```

We are interested in the accuracy score and the confusion matrix.

The confusion matrix provides significant insights into how the trained model is performing and what the weaknesses are.

Here, I have developed a function for drawing the confusion matrix.

```
def draw_confusion_matrix(y, y_pred, title):
    cm = confusion_matrix(y, y_pred, labels=np.unique(y))
    classes = np.unique(y)

    plt.figure(figsize=(8,6))
    plt.imshow(cm, interpolation='nearest', cmap=plt.cm.Blues)
    plt.title(title)
    plt.colorbar()

    tick_marks = np.arange(len(classes))
    plt.xticks(tick_marks, classes, rotation=45)
    plt.yticks(tick_marks, classes)

    thresh = cm.max() / 2
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            plt.text(j, i, cm[i, j],
                     horizontalalignment="center",
                     color="white" if cm[i, j] > thresh else "black")

    plt.ylabel('True Label')
    plt.xlabel('Predicted Label')
    plt.tight_layout()
    plt.show()
```

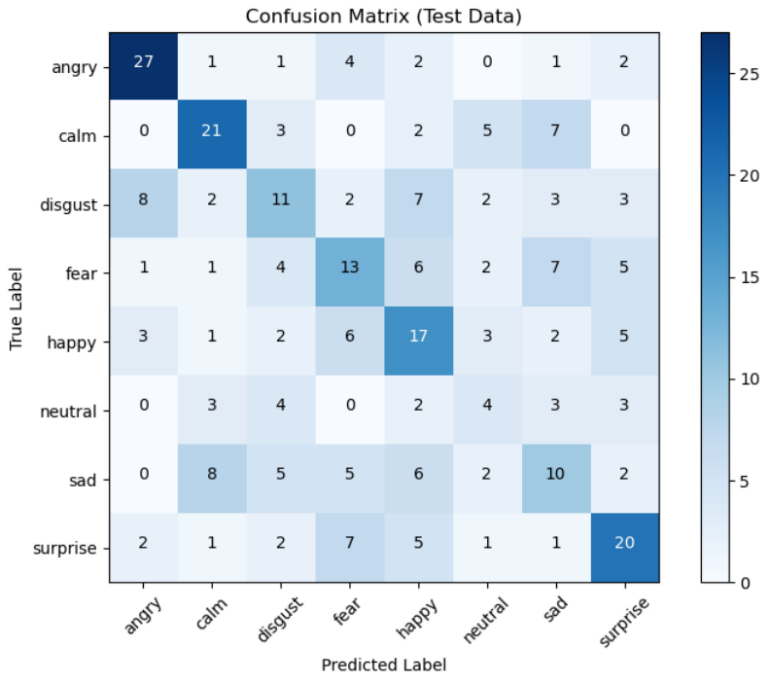

Figure 4: Confusion Matrix for LDA Model (Testing Data)

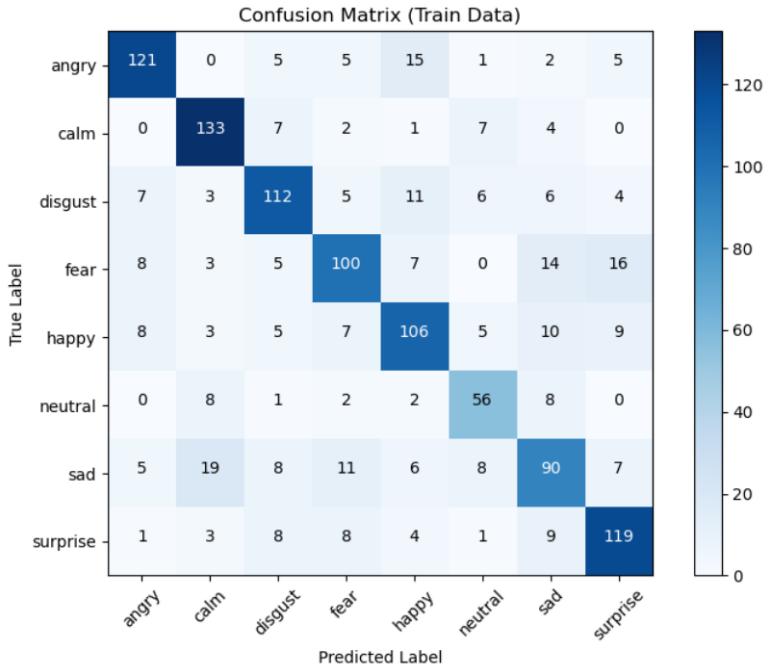

Figure 5: Confusion Matrix for LDA Model (Training Data)

And here are the accuracy results for the testing and training data using the LDA model.

Testing Data	42.7%
Training Data	72.6%

Table 1: LDA Model Accuracy Results

6.1.1 Key Insights

The accuracy results are not pleasing at all.

The model seems to achieve an acceptable accuracy result for the data it's been trained on, but the accuracy result for the testing data is not satisfactory at all.

Even though we are only getting a 42.7% accuracy for the testing data, it's much better than randomly guessing the class for each data sample, which would give us an estimated accuracy of 12.5% for the 8 classes we have.

Another thing to consider is the confusion matrix for both the testing and the training data.

Even though the accuracy results are not satisfactory at all, they seem reasonable and do not come from any bugs or problems in our approach or coding.

The thing is, our LDA model is confusing classes that real human beings also have a pretty high chance of confusing.

For example, the model is confusing *Sad* for *Calm*, or it's confusing *Fear* for *Surprise*.

6.2 SVM (No Kernel)

For this part, we change our machine learning model to **SVM** or **Support Vector Machine**.

The SVM algorithm tries to estimate hyperplanes in the n-dimensional space (n being the number of features) that best separates the different classes we have.

The following code is for training the SVM model using our training data.

Note that we will not be tweaking the parameters of our models and will be using the default values, because we are just evaluating our models to find the best one, and then we tweak the parameters to get the best performance possible.

```
svm_model = SVC(C=1, gamma='scale')
svm_model.fit(X_train, y_train)

y_pred_test = svm_model.predict(X_test)
y_pred_train = svm_model.predict(X_train)
```

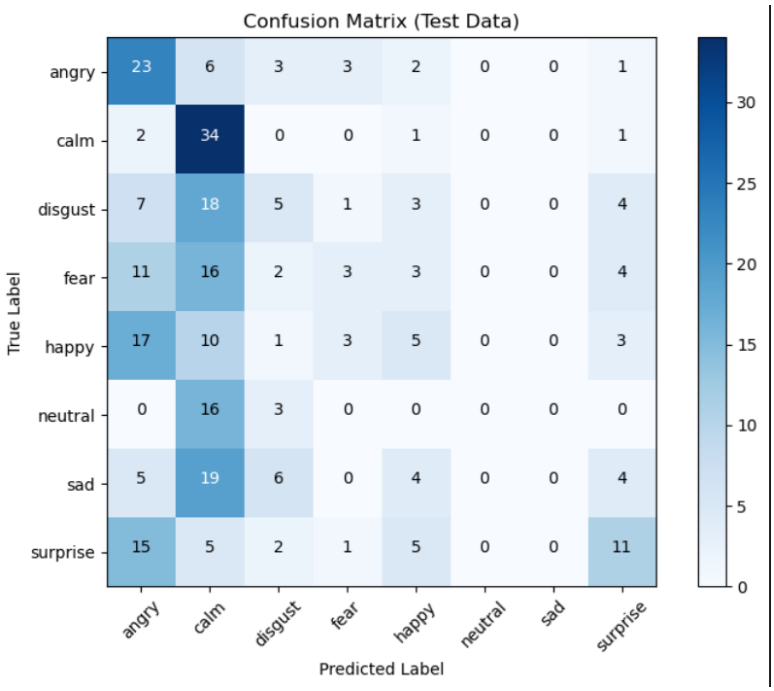

Figure 6: Confusion Matrix for SVM Model (No Kernel, Testing Data)

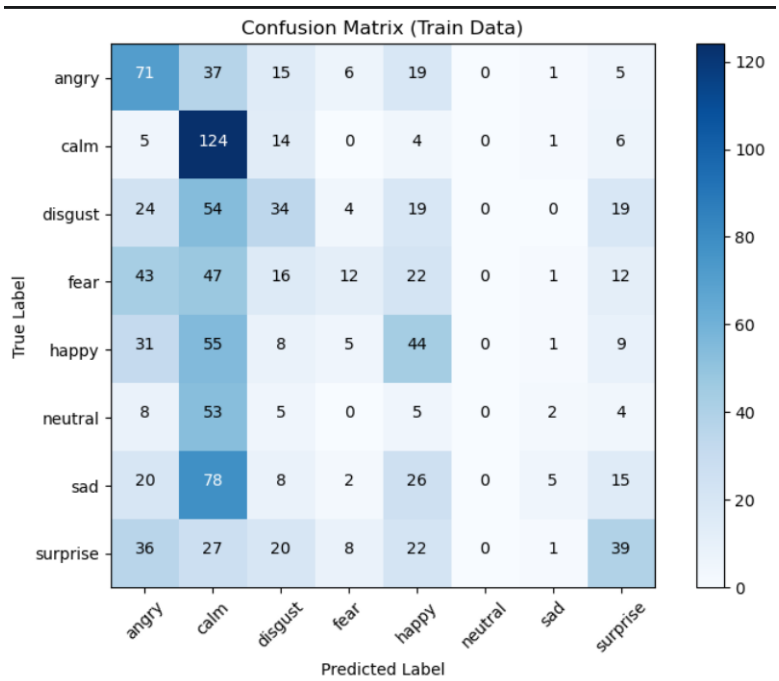

Figure 7: Confusion Matrix for SVM Model (No Kernel, Training Data)

Testing Data	28.1%
Training Data	28.5%

Table 2: SVM Model (No Kernel) Accuracy Results

6.2.1 Key Insights

The accuracy results are even worse than the LDA model.

An accuracy of 28% for both the training and the testing data might be an indicator of a fundamental problem with our model training.

Also by looking at our confusion matrix for both the testing and training data, we can see that many of our data samples have had their label confused for *Calm*, which might be an indicator of a fundamental problem in our approach, not just a simple bug.

The problem might be caused by not using a kernel in training our SVM model.

When not using a kernel, our SVM model is just a linear classifier.

It can only estimate the best hyperplanes (normal, straight ones) to separate the classes.

The solution might be using the RBF Kernel, which maps the data into a space of infinite dimension.

That means the curves separating our 8 classes can be of any shape and form.

6.3 SVM (RBF Kernel)

Now we will train our SVM model using the RBF kernel.

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

svm_rbf = SVC(kernel='rbf', C=1, gamma='scale')
svm_rbf.fit(X_train_scaled, y_train)

y_pred_test = svm_rbf.predict(X_test_scaled)
y_pred_train = svm_rbf.predict(X_train_scaled)

accuracy = accuracy_score(y_test, y_pred_test)
```

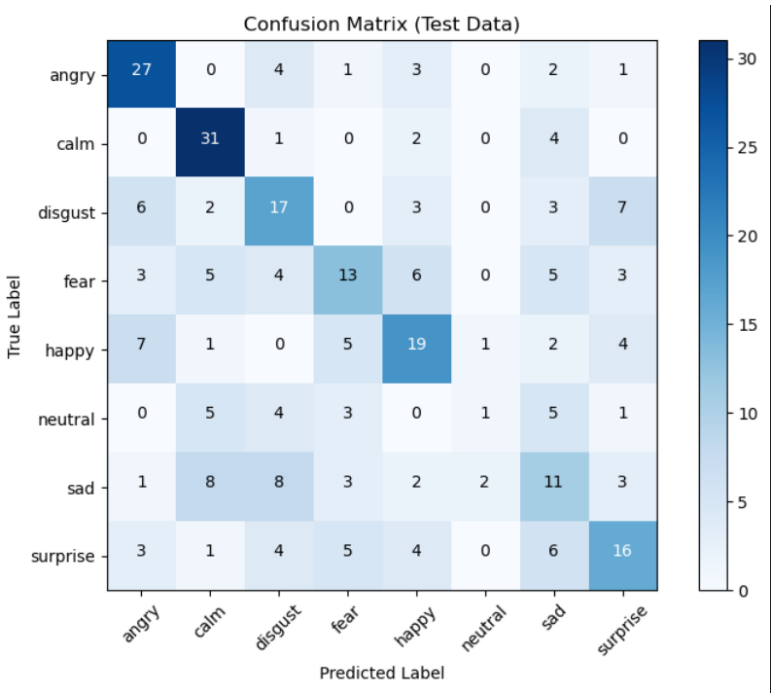

Figure 8: Confusion Matrix for SVM Model (RBF Kernel, Testing Data)

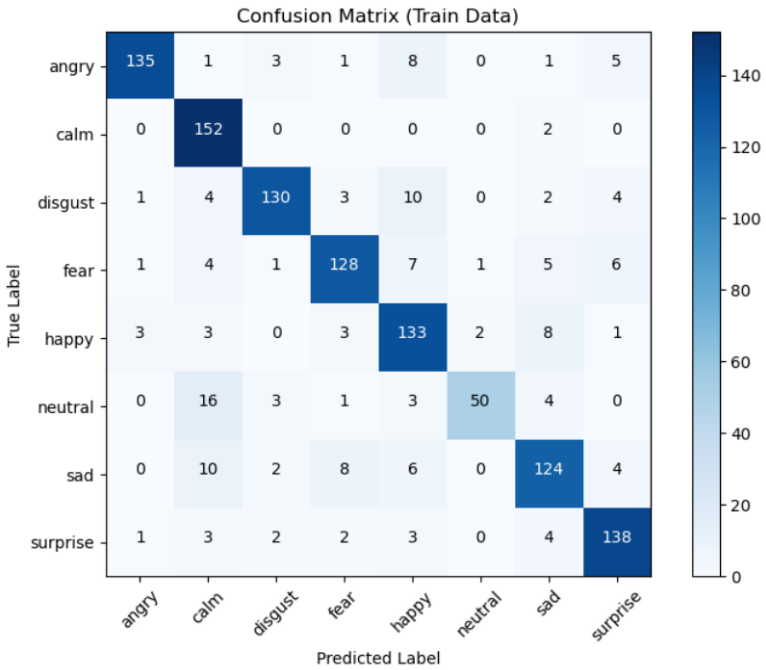

Figure 9: Confusion Matrix for SVM Model (RBF Kernel, Training Data)

Testing Data	46.8%
Training Data	85.9%

Table 3: SVM Model (RBF Kernel) Accuracy Results

6.3.1 Key Insights

Our confusion matrix makes a lot more sense now compared to the confusion matrix from the SVM model with no kernel.

We can see that emotions like *Calm* and *Sad* are being confused for each other which makes a lot more sense compared to the previous results where almost all of the data samples had their labels confused with *Calm*.

Our accuracy results are also higher for both the testing and the training data compared to both the LDA and the SVM (No Kernel) models.

7 The Problem

After training our models and getting the accuracy results, here is the best result we have achieved so far.

Testing Data	46.8%
Training Data	85.9%

Table 4: Best Accuracy Results

These numbers are not satisfactory at all.

Even though these are not bad results considering the fact that if we had labled the data samples randomly, we would have had an accuracy of 12.5% compared to an accuracy of 46.8% that we currently have for our test data.

These results might be caused because of these:

- **Too Many Features:** These results might be caused by having too many features feeded into our model.

  Having too many features slows the model training down, but that's not the real problem.

  If we feed the model the features that have no correlation with the label for our data samples, or features that just have a lot of correlation with themselves, the model will get confused.

  The best way to overcome this problem is to use **Dimension Reduction Algorithms**.

- **Low Number of Data Samples:** Considering the fact that we have 8 classes in our dataset, having only 1440 data samples is not enough to train our machine learning models at all.

  Having only 1440 data samples for 8 classes means that we only have 180 data samples for each of our classes, which is not enough at all.

  We can use 2 different approaches to solve this problem.

  Reducing the Number of Classes: One approach can be to reduce the number of our classes.

  That means we divide our 8 classes (emotions) into 2 emotion classes. *Positive* emotions and *Negative* emotions.

  Using Data Augmentation: Another approach can be to perform data augmentation techniques to our existing dataset to increase the number of our data samples.

7.1 What is Data Augmentation?

Data augmentation increases the number of data samples by changing and altering the already existing data in a way that does not change the class at all.

Different data augmentation techniques can be used for different types of data, but we are only concerned about audio signal data.

For audio signals, the following methods can be used to generate new data from the already existing dataset without altering the classification of the data samples.

- Adding Noise
- Stretching the Signal in the Time Domain
- Changing the Volume of the Audio Signals

Using these augmentation techniques together, can enable us to increase our data samples by a factor of 10 or even more.

Now we perform these methods one by one to see which one works the best.

8 Class Reduction

We first start by reducing the number of our classes from 8 to 2 without changing our dataset.

In fact, instead of generating new data for our 8 classes, we are reducing the number of our classes to better match the number of our samples.

```
def extract_emotion_type(emotion):
    emotion_map = {
        'neutral': 'positive',
        'calm': 'positive',
        'happy': 'positive',
        'sad': 'negative',
        'angry': 'negative',
        'fear': 'negative',
        'disgust': 'negative',
        'surprise': 'positive'
    }

    return emotion_map[emotion]
```

Now we train all of the machine learning models we used in the previous sections using the new labels (*Positive* and *Negative*).

```
lda = LinearDiscriminantAnalysis()
lda.fit(X_train, y_train)
y_pred_test = lda.predict(X_test)
y_pred_train = lda.predict(X_train)

svm_model = SVC(C=1, gamma='scale')
svm_model.fit(X_train, y_train)
y_pred_test = svm_model.predict(X_test)
y_pred_train = svm_model.predict(X_train)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
svm_rbf = SVC(kernel='rbf', C=1, gamma='scale')
svm_rbf.fit(X_train_scaled, y_train)
y_pred_test = svm_rbf.predict(X_test_scaled)
y_pred_train = svm_rbf.predict(X_train_scaled)
```

Since the confusion matrix gives us little to no insight for just 2 classes, here we only report the accuracy results for each of our models.

Testing Data	64.9%
Training Data	76.8%

Table 5: LDA Model Accuracy Results

Testing Data	54.5%
Training Data	55.9%

Table 6: SVM Model (No Kernel) Accuracy Results

Testing Data	70.8%
Training Data	90.7%

Table 7: SVM Model (RBF Kernel) Accuracy Results

8.1 Key Insights

At the first glance, these results might seem to be much better.

Especially if we consider the 71% accuracy for the SVM model with RBF kernel, it seems to be a lot of improvement.

But that's unfortunately not the case.

You see, if we had decided randomly about the label for each of our testing samples, we would have had an accuracy of 50%.

So 71% compared to 50% doesn't seem to be that much of an improvement.

In fact, an accuracy of 47% compared to 12.5% when labeling randomly is much better than an accuracy of 71% compared to 50% when labeling randomly.

So it seems like reducing the number of our classes didn't help at all.

9 Dimension Reduction

Now we perform a dimension (Feature) reduction algorithm called **PCA** to reduce the dimension of our space.

We have chosen the parameters of the PCA model to maintain 95% variance.

Now we feed the reduced features into our model to see how the results change.

Note that we are only doing this for SVM (No Kernel) and SVM (RBF Kernel) models, because the LDA model applies dimension reduction by default.

```
pca = PCA(n_components=0.95)

X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)

svm = SVC(C=1, gamma='scale')
svm.fit(X_train_pca, y_train)

y_pred_test = svm.predict(X_test_pca)
y_pred_train = svm.predict(X_train_pca)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

pca = PCA(n_components=0.95)

X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

svm_rbf = SVC(kernel='rbf', C=1, gamma='scale')
svm_rbf.fit(X_train_pca, y_train)

y_pred_test = svm_rbf.predict(X_test_pca)
y_pred_train = svm_rbf.predict(X_train_pca)
```

Testing Data	21.5%
Training Data	23.1%

Table 8: SVM Model (No Kernel) Accuracy Results

Testing Data	46.5%
Training Data	83.7%

Table 9: SVM Model (RBF Kernel) Accuracy Results

9.1 Key Insights

We can see that applying the dimension reduction algorithm PCA had a negative effect on our accuracy results, if any.

So it seems like we have to dump this idea as well.

10 Data Augmentation

The last thing we can do to increase our models accuracy, is to increase our dataset size using data augmentation techniques for audio signals.

Here we state the techniques we use to change the audio samples without altering their classification.

10.1 Stretch in the Time Domain

By stretching or compressing our audio signals in the time domain, we are not changing the classification of our audio signal, but we are altering it in a harmless way.

This helps us increase our dataset in size to better train our machine learning models.

```
def stretch_time(audio_file, rate):
    n_samples = audio_file.shape[0]
    new_length = int(n_samples / rate)

    old_indices = np.arange(n_samples)
    new_indices = np.linspace(0, n_samples - 1, new_length)

    stretched_audio = np.interp(new_indices, old_indices, audio_file)

    return stretched_audio
```

Note that for stretching the audio signals in the time domain, we have used the linear interpolation method from the NumPy library.

Even though it could have easily been implemented, we decided to use the one implemented in the NumPy library.

10.2 Adding AWGN Noise

10.2.1 What is AWGN Noise?

AWGN or **Additive White Guassian Noise** is a set of independent gussian random variables (it's a countable set for discrete time signals), that have a mean value of 0.

In fact, if $w[n]$ is our discrete time noise signal, we could state that:

$$\forall k \in \mathbb{N} \rightarrow w[k] \sim \mathcal{N}(\mu, \sigma^2)$$

where $\mu = 0$.

And we can also state that:

$$\forall m, n \rightarrow w[m] \perp\!\!\!\perp w[n]$$

The additive part means that this noise has to be added to the original signal.

So for implementing the AWGN noise in Python using NumPy, we have to generate as many independent gussian random variables with a mean value of 0 as we need, and make a discrete time signal out of these generated random values.

That will be our noise signal which has to be added to the original signal.

```
def add_noise(audio_file, value):
    n_samples = audio_file.shape[0]
    std = value * np.max(np.abs(audio_file))

    noise = np.random.normal(0, std, n_samples)
    audio_file_noisy = noise + audio_file

    return audio_file_noisy
```

10.3 Adjusting the Audio Signal Volume

Another way to change our audio signal without altering it's classification is to change it's volume.

Since we normalize our audio signals in amplitude with our extracting any features, this should cause no problems. Just giving us more sample data to train our models with.

```
def change_volume(audio_file, ratio):
    return ratio * audio_file
```

Now we merge all of these augmentation techniques into a single Python method.

```
    rate = np.random.uniform(0.9, 1.1)
    output_audio_file = stretch_time(audio_file, rate)

    if add_noise_bool:
        noise_level = np.random.uniform(0.03, 0.05)
        output_audio_file = add_noise(output_audio_file, noise_level)

    vol_ratio = np.random.uniform(0.85, 1.15)
    output_audio_file = change_volume(output_audio_file, vol_ratio)

    return output_audio_file
```

To make sure that these augmentations do not alter the audio signals too much that make the audio signal unrecognizable even by humea ears, we aggregate the sample audio signals from the beginning and save them and listen to them to make sure everything is fine.

Here is how our aggregated signals look like. with and without AWGN noise.

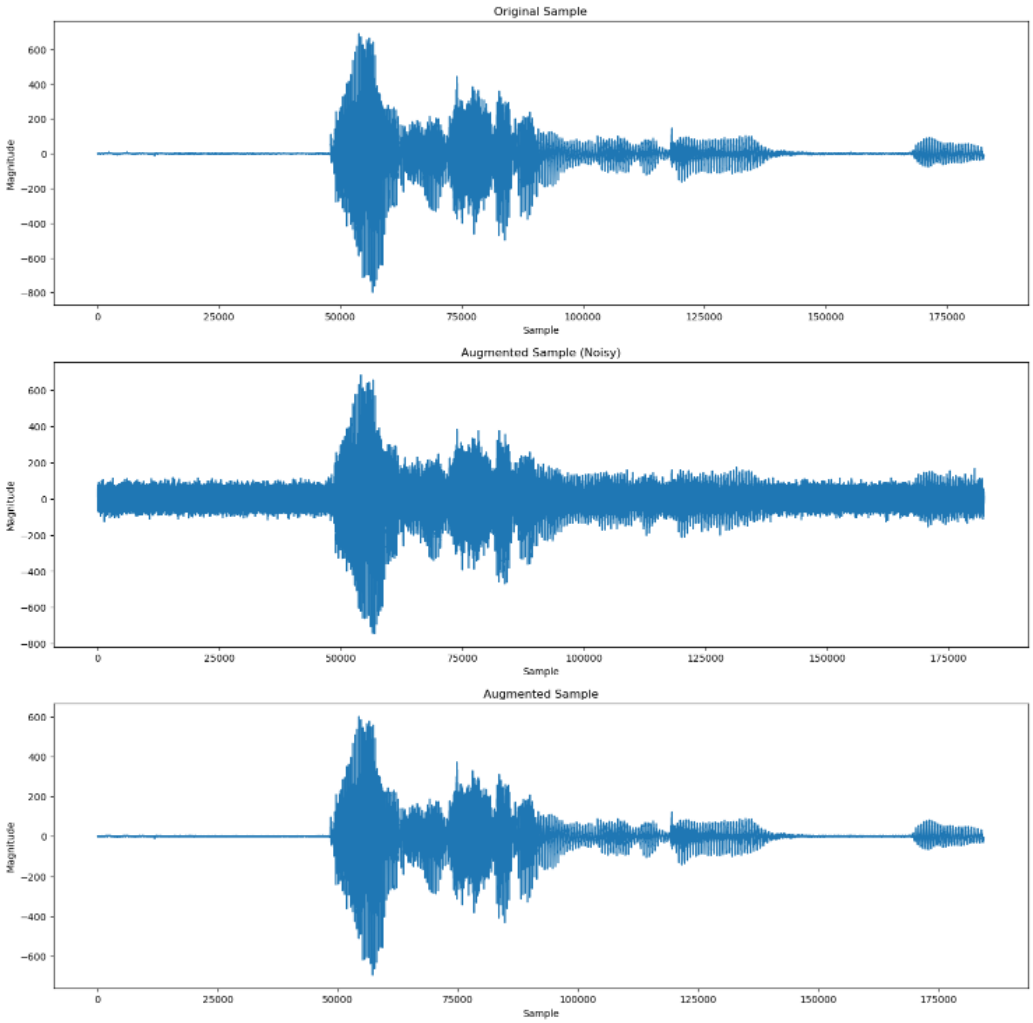

Figure 10: Aggregated Audio Signals from The Original Sample

It's clear that the audio signals are not altered fundamentally and the classification for our altered data is not changed at all.

Now the only thing that remains to be done is to read the original dataset, and make 10 new data samples from each audio file. 5 with AWGN and 5 without.

That means we are increasing our dataset size by a factor of **11**, which is great for 8 classes.

```
files = []

folder_path = './dataset'
all_file_names = [os.path.join(folder_path, f) for f in os.listdir(folder_path)]
count = 0
all_count = len(all_file_names)
progress_name = 'Reading Data'

for file_name in all_file_names:
    progress_bar(progress_name, count, all_count)
```

```
    Fs, original_audio = wavfile.read(file_name)

    if original_audio.ndim == 2:
        original_audio = original_audio.mean(axis=1)

    file = get_aggregated_features(original_audio, Fs)
    file['file_name'] = file_name
    emotion = extract_emotion_label(file_name)
    file['emotion'] = emotion

    files.append(file)

    for i in range(5):
        file = get_aggregated_features(augment_data(original_audio, True), Fs)
        file['file_name'] = file_name + ' - Augmented Noisy ' + '(' + str(i+1) + ')'
        file['emotion'] = emotion
        files.append(file)

        file = get_aggregated_features(augment_data(original_audio, False), Fs)
        file['file_name'] = file_name + ' - Augmented ' + '(' + str(i+1) + ')'
        file['emotion'] = emotion
        files.append(file)

    count += 1

progress_bar(progress_name, all_count, all_count)
```

11 Re-training the Machine Learning Models

After performing data aggregation and increasing the size of our dataset by a factor of 11, it's time to train our machine learning models again and see how the results have changed.

Since the code for re-training our models is exactly the same as before, we only include the accuracy results in this document.

The code and confusion matrices for each of the trained models are available in the Jupyter Notebook attached to this document.

Testing Data	54.0%
Training Data	57.1%

Table 10: LDA Model Accuracy Results

Testing Data	31.2%
Training Data	31.9%

Table 11: SVM Model (No Kernel) Accuracy Results

Testing Data	79.8%
Training Data	91.0%

Table 12: SVM Model (RBF Kernel) Accuracy Results

11.1 Key Insights

We can see that augmenting the original dataset had the most effect on the SVM (RBF Kernel), which is the best result we have had so far with an accuracy of 79.8% for our testing data.

I also tried the PCA dimension reduction algorithm for the SVM (RBF Kernel) model, but as I had tried before, it actually had a negative effect if any.

I also include the confusion matrices for the SVM (RBF Kernel) model in this document. The confusion matrices for other models can be found in the **Jupyter Notebook** attached to this document.

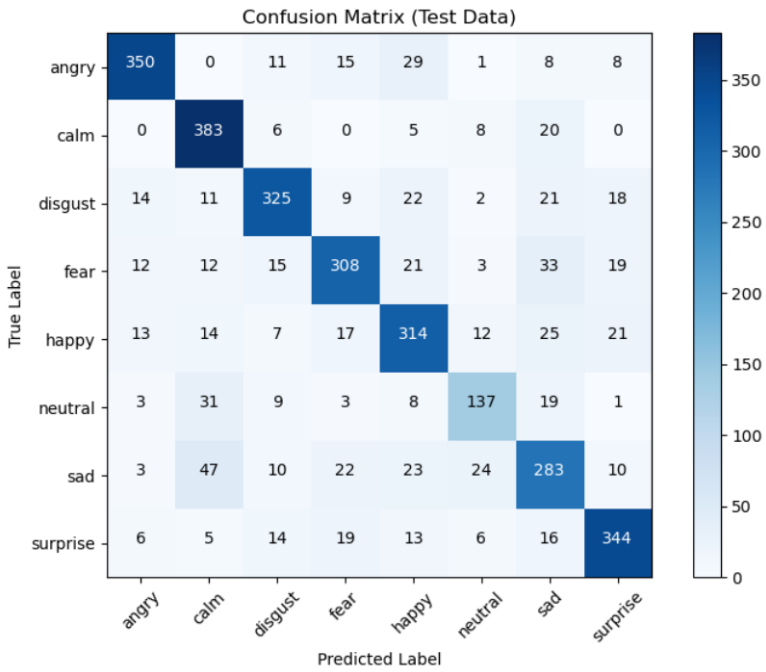

Figure 11: Confusion Matrix for SVM Model (RBF Kernel, Testing Data)

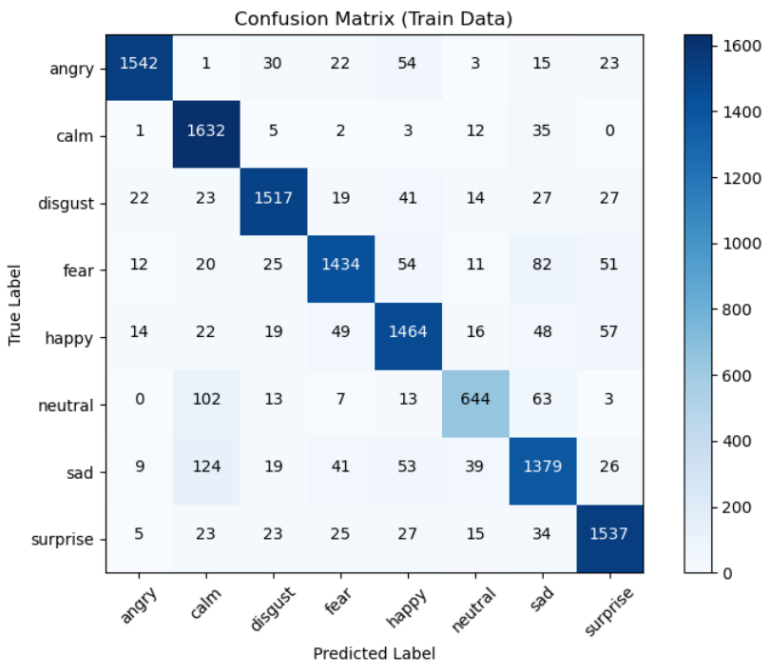

Figure 12: Confusion Matrix for SVM Model (RBF Kernel, Training Data)

We can observe astonishing results in the confusion matrices for both the testing and training data, which lets us make the conclusion that the SVM (RBF Kernel) is the best machine learning model for our cause.

12 Tuning Model Parameters

Now that we know what the best performing machine learning model for our cause is, it's time to tune the model parameters to achieve the best performance possible.

For this, we use 2 machine learning concepts / tools called the **Grid Search** and **Cross Validation**.

12.1 What is Grid Search?

Grid Search lets us make a grid of all of the hyperparameters of our model and test every combination of them to find the best performing one.

12.2 What is Cross Validation (CV)?

CV or Cross Validation is a technique that enables us to estimate the performance of our model on unseen data.

For this, we split the training data into k folds, then train our model on the first fold of training data and estimate it's performance using the second fold of training data.

This is performed k times, that means that the model is trained k times. Then we take the average performance across all folds and report that.

Using this method, we get a pretty good estimation of how well the model is tuned with the chosen parameters from the parameter grid without exposing the test data to the models.

We have decided to have 5 folds ($k = 5$) for our CV and have a total number of 30 possible parameter pairs.

That means that our SVM (RBF Kernel) model has to be trained a total number of 150 times so we can find the best performing model (parameters).

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

param_grid = {
    'C': [0.1, 1, 10, 50, 100],
    'gamma': ['scale', 0.01, 0.05, 0.1, 0.5, 1],
    'kernel': ['rbf']
}

grid = GridSearchCV(SVC(), param_grid, cv=5,
scoring='accuracy', verbose=1, n_jobs=-1)

grid.fit(X_train_scaled, y_train)

print("Best parameters:", grid.best_params_)
print("Best CV accuracy:", grid.best_score_)

best_svm = grid.best_estimator_
```

```
y_pred_test = best_svm.predict(X_test_scaled)
y_pred_train = best_svm.predict(X_train_scaled)
accuracy_test = accuracy_score(y_test, y_pred_test)
print("Test accuracy:", accuracy_test)
```

After finding the best performing model, here are our results.

Best CV Accuracy	88.1%
Testing Data Accuracy	91.7%

Table 13: SVM Model (RBF Kernel) Accuracy Results with the Best Performing Parameters

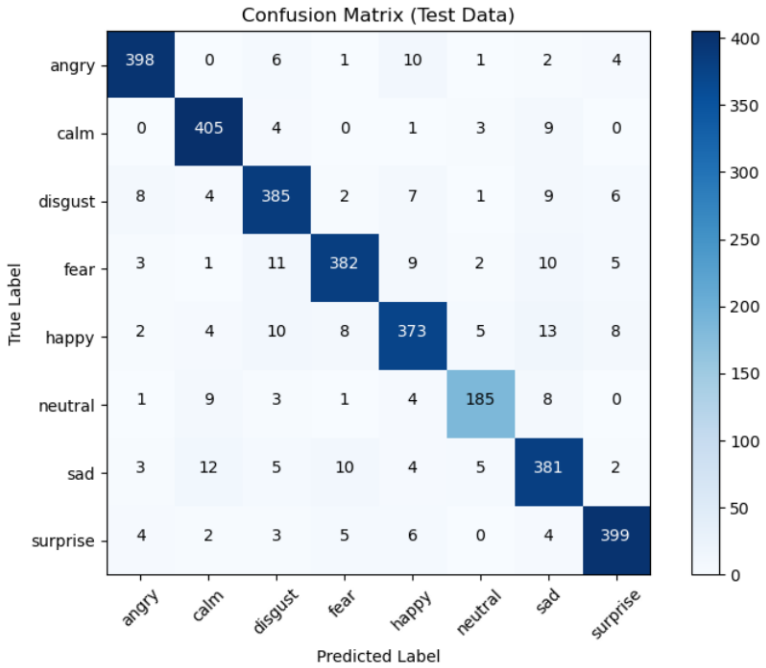

Figure 13: Confusion Matrix for SVM Model with the Best Performing Parameters (RBF Kernel, Testing Data)

12.3 Key Insights

The final results for our machine learning model is astonishing.

An accuracy of 91.7% considering the fact that we have 8 classes and not a lot a data in our dataset, is pretty impressive.

I think this number is the one to settle down with, it also seems like no furthur optimizations can be done to increase this number (at least not any major ones).

13 Generalization

Now we want to observe how well our model predicts and classifies audio signals outside the dataset we used to train our models.

The thing is, all of our training audio signals have been recorded in a controlled studio environment, and an accuracy score of 91% for the test data taken from the original dataset, does not necesserily imply that our model can classify any audio signal.

So we start by classifying 5 sample audio files, 2 from our own recordings, 3 from the original dataset (**RAVDESS**).

We are also going to use our best performing model so far.

```
file_names = ['./self_recorded/sad.wav',
              './self_recorded/happy.wav',
              './sample.wav',
              'augmented_noisy.wav',
              'augmented.wav']

labels = ['sad', 'happy', 'calm', 'calm', 'calm']

files = []

for file_name in file_names:
    Fs, original_audio = wavfile.read(file_name)

    if original_audio.ndim == 2:
        original_audio = original_audio.mean(axis=1)

    file = get_aggregated_features(original_audio, Fs)
    file['file_name'] = file_name

    files.append(file)

features = [feature for feature in files[0].keys() if feature != 'file_name']

X_test_new = [[file[k] for k in features] for file in files]
X_test_scaled_new = scaler.transform(X_test_new)

predicted_labels = best_svm.predict(X_test_scaled_new)

print(f'Actual Labels: {labels}')
print(f'Predicted Labels: {predicted_labels}')
```

Here are the results for our best performing model.

```
Actual Labels: ['sad', 'happy', 'calm', 'calm', 'calm']

Predicted Labels: ['disgust' 'disgust' 'calm' 'calm' 'calm']
```

13.1 Key Insights

We can clearly observe that our SVM model is labeling all of the self-recorded audio files as *Disgust*.

This behavior is due to a concept in machine learning called **Feature Distribution Mismatch**.

13.1.1 What is Feature Distribution Mismatch?

Differences in recording environment, sampling rate, amplitude, and audio length cause the extracted features to fall outside the regions seen during training.

SVMs, being sensitive to feature distributions, map these out-of-distribution inputs to the nearest support vectors, resulting in a single-class prediction.

Even the smallest difference in mic quality or environment can cause the SVM model to be confused.

That's why we achieve a high accuracy value for test data from the original dataset, but all of the audio signals from sources outside that dataset get mapped to a single class (*Disgust* in this case).

Since the LDA model is less sensitive to this, it might have better generalization even though it had lower accuracy score for the original test data.

```
predicted_labels = lda.predict(X_test_new)

print(f'Actual Labels: {labels}')
print(f'Predicted Labels: {predicted_labels}')
```

Here are the results:

```
Actual Labels: ['sad', 'happy', 'calm', 'calm', 'calm']

Predicted Labels: ['neutral' 'surprise' 'calm' 'calm' 'calm']
```

13.2 Key Insights

Even though the LDA model scored much lower in the original test data, it seem to have a better generalization capability than our SVM model.

Even though it has not predicted the labels for our self-recorded audio files correctly, it at least has not mapped all of them into a single class.

Also, the predicted labels are not that far off.

The model confused *Sad* for *Neutral* and *Happy* for *Surprise*, even actual human being could have made the same mistake when listening to these files.

So it seems like out LDA model has a better generalization capability than our SVM model, so we optimize our LDA model parameters for the best performance using cross validation.

```
lda = LinearDiscriminantAnalysis()

param_grid = [
```

```
    {'solver': ['svd']},
    {
        'solver': ['lsqr', 'eigen'],
        'shrinkage': [None, 'auto', 0.1, 0.3, 0.5, 0.7]
    }
]

grid = GridSearchCV(
    lda,
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

grid.fit(X_train, y_train)

print("Best params:", grid.best_params_)
print("Best CV accuracy:", grid.best_score_)
```

Now we classify the general data using the best performing LDA model.

Here are the results.

```
Actual Labels: ['sad', 'happy', 'calm', 'calm', 'calm']

Predicted Labels: ['neutral' 'surprise' 'calm' 'calm' 'calm']
```

13.2.1 Key Insights

Even though our model is still not predicting the exact label for the self-recorded audio signals, it's confusing the actual labels for labels that make sense.

For the final generalization test, we take a part of the **EmoDB** dataset, which is a speech dataset in the German language, and use it to evaluate how well our model generalizes.

Since the code for reading the data and extracting the features is the same as before, we do not include it in this document, as stated before, it's available in the Jupyter Notebook attached to this document.

Here are the results for the EmoDB dataset.

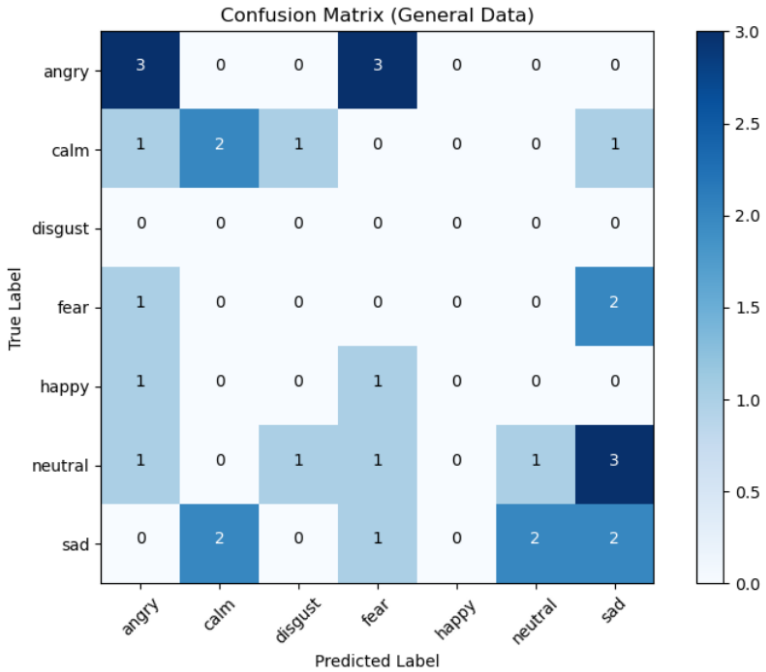

Figure 14: Confusion Matrix for LDA Model with the Best Performing Parameters (General Data)

Even though we only got a 16% accuracy score, our confusion matrix states that the model is confusing actual labels with labels that actually make sense, which might or might not be fine depending on the task we are trying to use this system in.

14 Conclusion

Even though our performance was astonishing for the original dataset that we used to train our models, but our models failed at generalization,

I think that this system is still completely functionable enough for tasks like voice chat bots which only need a rough estimation of what your feeling is.

At the end of the day, using manual feature engineering for a complex machine learning task like this has little to no practicality value, it's just for understanding the concepts in feature engineering for audio signals and the underlying principle of machine learning models.

If we actually wanted to develop a reliable audio-based emotion detection system, we should have used **Deep Learning Models** instead of machine learning models, which is the industry best practice for this cause.